Architectural Justification

Scenario: Self-Service Coffee Kiosk System

> *We chose **Layered Architecture** for the Self-Service Coffee Kiosk System.*

Architectural Choice

The kiosk is split into three layers. Each layer calls only the layer directly below it, through a named interface: **Presentation** (Touchscreen UI), **Business Logic** (Order Manager, Payment Service) and **Data & Hardware Access** (Menu & Pricing Repository, Receipt Printer Service). The card reader, the thermal printer and the bank's payment gateway sit outside the software and are reached only through interfaces in the lower layers.

Style	Fit for this kiosk
Layered ✓	A single self-contained machine with a clear UI → logic → data/hardware flow. Simple to build, test and maintain.
Microservices	Independent deployment and scaling are unnecessary for one kiosk with 6 menu items. The network overhead and operational cost outweigh the benefits.
Client-Server	A central server would add a single point of failure. If the café's network went down, the kiosk could not even show the menu.

Two Reasons for the Choice

1. **Hardware isolation.** The kiosk depends on a touchscreen, a receipt printer and a card reader, which are hardware a café may replace. Each device sits behind its own lower-layer interface (*IPrinterPort*, *ICardReader*). Swapping a printer model therefore changes only the Receipt Printer Service, and the Order Manager and UI stay untouched.
2. **Requirements change mostly in one layer.** Likely changes include adding a coffee type, a Medium size or new prices. These touch only the Menu & Pricing Repository and its data (*IMenuData*). A UI redesign touches only the Presentation layer. This separation of concerns suits a small, stable system that is maintained over years.

Security Advantage

The layers create a **controlled path for card data**. The Touchscreen UI never sees card details. It can only call *IOrderEntry*, and the Order Manager receives just a *PaymentResult* through *IPayment*. Raw card data stays inside the Payment Service, which reads a token from the PIN pad (*ICardReader*) and talks to the bank only over HTTPS/TLS (*IGatewayAPI*). A bug or tampering in the customer-facing layer therefore cannot leak card numbers. This shrinks the PCI-DSS scope to a single component.

Performance Benefit

All internal layers run **in-process on the kiosk**, so menu look-ups and order totals are local function calls and SQLite queries. They take well under a millisecond, with no network hops of the kind microservices would add. The touchscreen therefore responds instantly even during a busy café rush. The only remote call in the system is the single card authorisation at checkout.